1 Trabajo Práctico – Sistemas de Computación

Integrantes:

- Brigido Tomás
- Siñuka Javier
- Tosolini Agustín

1 **Trabajo Práctico – Sistemas de Computación** **1**

1.1 Introducción 3

1.2 Desarrollo 4

1.2.1 Benchmarking de hardware 4

1.2.2 Comparativa de Compilación de Linux Kernel 6

1.2.3 Aceleración Procesador Ryzen 9 7950X vs AMD Ryzen 9 5900X 12-Core 7

1.2.4 Analizando el rendimiento de un programa con la herramienta gprof 8

1.2.5 Prueba de ESP32 9

1.3 Conclusión 12

1.1 Introducción

El presente trabajo práctico se enmarca en el estudio del rendimiento y la performance de los sistemas de computación, con el objetivo de trasladar conceptos teóricos a un contexto de análisis aplicado. En particular, la actividad propone observar el comportamiento de una computadora desde dos perspectivas complementarias: por un lado, el uso de benchmarks de terceros como herramienta de referencia para comparar alternativas de hardware y fundamentar decisiones técnicas; por otro, la utilización de herramientas de medición orientadas al análisis de performance de programas propios, con foco en el tiempo de ejecución y en el costo computacional de las funciones que componen una solución de software.

Desde el punto de vista de la ingeniería, ambas dimensiones resultan necesarias. El análisis de hardware mediante benchmarks permite interpretar métricas de desempeño en escenarios estandarizados y relacionarlas con criterios concretos de selección de componentes. A su vez, el estudio de la performance de código introduce una mirada más cercana al desarrollo de software, ya que no solo interesa que un programa funcione correctamente, sino también comprender cómo utiliza el tiempo de CPU, qué partes concentran el mayor costo de ejecución y qué oportunidades de optimización pueden identificarse a partir de evidencia experimental.

1.2 Desarrollo

En esta sección se analizan distintas métricas de rendimiento con el objetivo de vincular conceptos teóricos de Sistemas de Computacion con situaciones de uso concretas. El enfoque adoptado combina resultados de benchmarks sintéticos y de aplicaciones reales para observar cómo influyen variables como la cantidad de núcleos, la frecuencia efectiva, la jerarquía de memoria y el subsistema de almacenamiento sobre el desempeño global del sistema. De este modo, no solo interesa comparar valores numéricos, sino también interpretar qué representan esas mediciones y en qué contextos resultan relevantes.

1.2.1 Benchmarking de hardware

El benchmarking de hardware consiste en ejecutar pruebas estandarizadas para medir el comportamiento de un sistema frente a cargas de trabajo determinadas. Estas herramientas permiten obtener resultados reproducibles y comparables entre distintas plataformas, siempre que se consideren variables de contexto como el sistema operativo, la temperatura de operación, la configuración de energía y la versión del software utilizado. Su utilidad principal radica en que ofrecen una aproximación cuantitativa al rendimiento, lo cual facilita la evaluación de componentes y la toma de decisiones de compra o actualización.

Lista de Benchmarks
Speedometer 2.0
Cinebench
Geekbench
3DMark
Unigine Heaven
CrystalDiskMark
AIDA64
PCMark
Blender
HandBrake

1.1

La tabla de benchmarks reúne herramientas orientadas a distintos subsistemas del equipo. Algunas se enfocan en CPU y memoria, como `Cinebench`, `Geekbench` y `AIDA64`; otras apuntan al rendimiento gráfico, como `3DMark` y `Unigine Heaven`; mientras que `CrystalDiskMark` permite medir el comportamiento del almacenamiento en operaciones secuenciales y aleatorias. A su vez, `PCMark`, `Blender` y `HandBrake` resultan especialmente útiles porque modelan escenarios cercanos a tareas reales de productividad, renderizado y codificación de video, lo que permite complementar los benchmarks puramente sintéticos con casos de uso más representativos.

Actividad diaria	**Benchmark asociado**
Navegación web	Speedometer 2.0
Compilar, multitarea pesada	Cinebench
Uso general	Geekbench
FPS en juegos	3DMark
Estabilidad gaming	Unigine Heaven
Carga de apps/juegos	CrystalDiskMark
Multitarea (RAM)	AIDA64
Experiencia diaria	PCMark
Edición/render	Blender
Exportar videos	HandBrake

1.2

La segunda tabla realizada relaciona cada benchmark con una actividad diaria concreta. Esta asociación es importante porque evita interpretar una métrica fuera de contexto. Por ejemplo, para navegación web interesa la capacidad de respuesta en cargas basadas en JavaScript y renderizado del navegador, por lo que `Speedometer 2.0` es una referencia adecuada. En cambio, para compilación o multitarea pesada resultan más representativos benchmarks con alta utilización de CPU multinúcleo, como `Cinebench`. Del mismo modo, la experiencia de uso general no depende de un único componente, sino del equilibrio entre

procesador, memoria, almacenamiento y aceleración gráfica, razón por la cual `PCMark` y `Geekbench` aportan una visión más integral del sistema.

En términos metodológicos, estas tablas muestran que no existe un único benchmark capaz de describir completamente el rendimiento de una computadora. Cada prueba aísla o enfatiza determinados recursos del sistema, por lo que un análisis técnico correcto requiere combinar varias métricas y luego contrastarlas con la carga real de trabajo del usuario. Esto es especialmente relevante en entornos universitarios y profesionales, donde la elección del hardware debe justificarse a partir del tipo de tarea predominante y no solamente por un puntaje global.

1.2.2 Comparativa de Compilación de Linux Kernel

La compilación del kernel de Linux es una carga de trabajo ampliamente utilizada para evaluar rendimiento de CPU en escenarios reales, ya que combina procesamiento intensivo, paralelismo y acceso frecuente a memoria y archivos intermedios. A diferencia de un benchmark sintético, compilar el kernel representa una tarea concreta del ámbito de sistemas y desarrollo, por lo que el tiempo total de ejecución resulta una métrica especialmente valiosa para comparar procesadores en un contexto técnico.

Procesador	**Arquitectura**	**Núcleos/Hilos**	**Tiempo Aprox. (defconfig)**
Intel Core i5-13600K	Raptor Lake (13 Gen)	14 (6P + 8E) / 20	~72 – 76 segundos
AMD Ryzen 9 5900X	Zen 3 (Ryzen 5000)	12 / 24	~82 – 86 segundos

1.3

Los valores de la tabla indican que el Intel Core i5-13600K logra una compilación más rápida que el Ryzen 9 5900X en la configuración `defconfig`, pese a que este último dispone de una mayor cantidad de hilos homogéneos. Esto puede explicarse por varios factores: mejoras de IPC en arquitecturas más recientes, mayores frecuencias sostenidas en cargas de compilación y una estrategia híbrida de núcleos de rendimiento y eficiencia que permite distribuir tareas concurrentes de manera efectiva. En este tipo de carga no solo importa el número de hilos disponibles, sino también la capacidad de cada núcleo para resolver instrucciones por ciclo y alimentar adecuadamente la canalización de ejecución.

Además, la compilación del kernel es sensible al subsistema de memoria y al almacenamiento, ya que involucra múltiples archivos fuente, generación de objetos temporales y enlazado. Por ello, aunque el tiempo final suele tomarse como indicador del procesador, en realidad refleja el comportamiento conjunto de varios componentes del sistema. Esta observación es importante para evitar conclusiones simplificadas: un procesador con más núcleos no necesariamente compilará más rápido si su rendimiento por núcleo, sus latencias o su plataforma acompañante no son igualmente competitivos.

1.2.3 Aceleración Procesador Ryzen 9 7950X vs AMD Ryzen 9 5900X 12-Core

La comparación entre el Ryzen 9 7950X y el Ryzen 9 5900X permite observar cómo una mejora generacional no impacta de manera uniforme sobre todas las cargas de trabajo. El 7950X incorpora una arquitectura más reciente, mayor cantidad de núcleos y mejoras de frecuencia e IPC, lo que se traduce en una ventaja clara en tareas paralelizables. Por eso, en cargas como compilación, renderizado o multitarea intensiva, la aceleración real se aproxima a valores del orden de `1.5x` o incluso superiores, ya que estas aplicaciones pueden distribuir el trabajo entre un gran número de hilos de ejecución.

Área	**Aceleración real**
CPU general	~1.5x
Compilación	~1.5x
Render / edición	~1.5x – 1.7x
Gaming	~1.1x – 1.4x
Multitarea	hasta ~1.8x

1.4

Sin embargo, en gaming la mejora relativa es menor porque el rendimiento total del sistema depende también de la GPU, de la resolución utilizada y del motor del juego. Cuando la carga está limitada por la placa de video, el beneficio de un procesador más potente se reduce considerablemente. En cambio, en escenarios de productividad avanzada, donde el cuello de botella principal está en CPU y memoria, el salto entre ambas generaciones resulta mucho más evidente. Esto demuestra que la evaluación de una actualización de hardware debe hacerse en función del perfil de uso: para cargas profesionales o de

desarrollo intensivo, el 7950X ofrece una mejora sustancial; para juegos, la relación costo-beneficio puede ser menos marcada si el resto del sistema no acompaña.

1.2.4 Analizando el rendimiento de un programa con la herramienta gprof

A continuación mostramos en detalle los componentes de las computadoras de cada uno de los integrantes del grupo para poder hacer un análisis de los resultados obtenidos.

Componente	**Tomás**	**Agustín**	**Javier**
Procesador	Intel® Core™ i7-10700F CPU @ 2.90GHz	11th Gen Intel® Core™ i5-1135G7 @ 2.40GHz	Intel® Core™ i7-10510U CPU @ 1.80GHz
Memoria RAM	16 GB DDR4	16 GB DDR4	16 GB DDR4
Almacenamiento	256 GB SSD NVMe M.2	256Gb GB SSD NVMe	500 GB SSD NVMe M.2

1.5

Dentro de la sección `scripts/results` se encuentran en detalle los resultados obtenidos por cada uno de los integrantes al utilizar la herramienta `gprof` sobre los scripts solicitados.

Tomás	**Javier**	**Agustín**

`total`
`s/call name`
`5.50 func1`
`3.35 func2`
`main`
`0.32 new_func1`
	`total`
	`s/call name`
	`7.57 func1`
	`4.60 func2`
	`0.45 new_func1`
	`main`
		`total`
		`s/call name`
		`9.33 func1`
		`5.66 func2`
		`main`
		`0.54 new_func1`

1.6

Como se puede observar, los resultados obtenidos para cada uno de los integrantes son distintos, ya que los componentes de cada computadora no son iguales. Podemos notar que

los procesadores más rápidos tienen tiempos más pequeños en cada función, lo que se traduce en un mejor rendimiento.

1.2.5 Prueba de ESP32

En esta sección se documenta la prueba realizada sobre la placa `ESP32`, con el objetivo de observar su comportamiento en un entorno de sistemas embebidos y relacionarlo con los conceptos de rendimiento analizados a lo largo del trabajo. Particularmente, interesa evaluar cómo responden sus recursos de procesamiento frente a una carga concreta, considerando las limitaciones propias de este tipo de plataformas en comparación con una computadora de propósito general.

Para la realización del estudio se utilizó la plataforma virtual `https://wokwi.com/projects/new/esp32`, ya que no contábamos con el microcontrolador físico. Esta herramienta permitió simular el comportamiento de una placa `ESP32` y ejecutar pruebas sencillas de rendimiento en un entorno controlado, lo cual resulta útil para aproximar mediciones experimentales sin necesidad de disponer del hardware real.

Con el objetivo de probar la velocidad del procesador, se utilizó el siguiente código, que configura la frecuencia de trabajo del CPU y luego ejecuta dos bucles de prueba: uno con operaciones enteras y otro con operaciones en punto flotante. De esta manera, se obtiene una medida aproximada del tiempo requerido por el microcontrolador para completar cargas de trabajo simples de naturaleza distinta.

```
/*
 * Prueba de rendimiento ESP32
 */

void setup() {
  // --- LINEA DE CONFIGURACION DE FRECUENCIA ---
  setCpuFrequencyMhz(8);
  // ---------------------------------------------

  Serial.begin(115200);
  delay(1000);

  uint32_t freq = getCpuFrequencyMhz();
  Serial.print("Frecuencia actual del CPU: ");
  Serial.print(freq);
```

```
  // Variables volatile para que el compilador no ignore los calculos
  volatile int32_t sumaEnteros = 0;
  volatile float sumaFloats = 0.0;

  long iteracionesInt = 12500000;
  long iteracionesFloat = 800000;

  Serial.println("\nIniciando prueba de 10 segundos aprox...");

  // --- PRUEBA DE ENTEROS ---
  unsigned long inicioInt = millis();
  Serial.println("Ejecutando bucle de enteros...");
  for (long i = 0; i < iteracionesInt; i++) {
    sumaEnteros += 1;
  }
  unsigned long finInt = millis();
  float tiempoInt = (finInt - inicioInt) / 1000.0;

  // --- PRUEBA DE FLOATS ---
  unsigned long inicioFloat = millis();
  Serial.println("Ejecutando bucle de floats...");
  for (long i = 0; i < iteracionesFloat; i++) {
    sumaFloats += 1.1;
  }
  unsigned long finFloat = millis();
  float tiempoFloat = (finFloat - inicioFloat) / 1000.0;

  // --- RESULTADOS ---
  Serial.println("---------- RESULTADOS ----------");
  Serial.print("Tiempo Enteros: "); Serial.print(tiempoInt); Serial.printl
  Serial.print("Tiempo Floats:  "); Serial.print(tiempoFloat); Serial.prin
  Serial.print("Tiempo Total:   "); Serial.print(tiempoInt + tiempoFloat);
  Serial.println("--------------------------------");
}

void loop() {
  // Nada por aqui
}
```

La prueba propuesta resulta adecuada para un análisis introductorio, ya que permite observar cómo impacta la frecuencia del procesador sobre el tiempo de ejecución de

ambos tipos de cálculo ejercen demandas distintas sobre la unidad de procesamiento. En consecuencia, este experimento constituye una forma simple pero representativa de estudiar el rendimiento del `ESP32` dentro del enfoque general del trabajo práctico.

A continuación, se presentan capturas correspondientes a ejecuciones realizadas con frecuencias de `4 MHz` y `8 MHz`, lo que permite comparar visualmente el efecto de la configuración del procesador sobre los tiempos reportados.

4mhz	**8mhz**

`Iniciando prueba de 10 segundos aprox...`
`Ejecutando bucle de enteros...`
`Ejecutando bucle de floats...`
`---------- RESULTADOS ----------`
`Tiempo Enteros: 10.38 seg`
`Tiempo Floats:  9.97 seg`
`Tiempo Total:  20.35 seg`
`----------`
	`Iniciando prueba de 10 segundos aprox...`
	`Ejecutando bucle de enteros...`
	`Ejecutando bucle de floats...`
	`---------- RESULTADOS ----------`
	`Tiempo Enteros: 8.33 seg`
	`Tiempo Floats:  8.00 seg`
	`Tiempo Total:  16.32 seg`
	`----------`

1.7

1.3 Conclusión

En este trabajo práctico se analizaron dos aspectos centrales del rendimiento en sistemas de computación: por un lado, el uso de benchmarks de hardware permitió relacionar métricas de desempeño con actividades concretas como navegación, gaming, compilación y multitarea; por otro, el uso de `gprof` hizo posible estudiar el comportamiento de programas propios y observar cómo las diferencias de hardware influyen en los tiempos de ejecución. En conjunto, el trabajo permitió comprender que el rendimiento depende tanto de los componentes del sistema como del tipo de tarea analizada y de las herramientas utilizadas para medirlo.

Un punto relevante del análisis es que los benchmarks no deben interpretarse como valores absolutos aislados, sino como indicadores que adquieren sentido cuando se vinculan con una carga de trabajo específica. La comparación entre pruebas orientadas a CPU, memoria, GPU y almacenamiento mostró que distintos usos del sistema exigen recursos diferentes y que, en consecuencia, la noción de "mejor hardware" depende del contexto de aplicación. Desde una perspectiva de ingeniería, esto implica que la evaluación de performance debe considerar siempre el escenario real de uso y no únicamente un puntaje general.

Asimismo, la comparación de tiempos de compilación y el estudio de aceleración entre procesadores de distintas generaciones permitieron observar que el desempeño está condicionado por múltiples variables: arquitectura interna, IPC, cantidad de núcleos e hilos, frecuencias sostenidas y eficiencia del subsistema de memoria. Finalmente, el análisis con `gprof` complementó esta visión desde el software, mostrando que el rendimiento no depende solo del hardware disponible, sino también de cómo un programa distribuye su costo computacional entre funciones. En síntesis, el trabajo permitió integrar una mirada de hardware y software sobre la performance, fundamental para analizar sistemas de computación con criterio técnico y fundamentado.